

AI Racer 视觉自动驾驶控制器

(按首字母) 董安愉 黄婧栩 李沐晓 曲清扬 许子涵

2026 年 6 月 19 日

摘要

本项目开发了一个纯视觉、无地图、无车速反馈的 AI Racer 控制器。仿真平台每帧传入左右双目图像与时间戳，控制器返回转向与速度两个标量值。控制流采用单向四级流水线。感知模块从双目图像中提取道路特征，估计模块将特征转换为车辆的几何状态（横向偏移、道路朝向、弯道曲率等），策略模块根据几何状态计算转向与速度指令，限幅模块确保输出在合法范围内。控制器按比赛类型拆分为两个相互隔离的参数配置，一套面向单车计时赛，另一套面向多车竞速赛。

单车在 complex 赛道完整行驶一圈，总用时 252.863 秒，最佳单圈 252.928 秒，全程零丢线、零硬碰撞。指令平均速度为 0.85（归一化值），满速指令占比 47.5%，低速爬行指令仅占 0.3%。多车评测使用同一套控制器同时驱动六辆车。在对手车扎堆时，早避让策略将本车陷入车堆的帧占比从 18.5% 压低至 0.7%，丢线率从 13% 降至 0，平均速度从 0.67 恢复至 0.85；分层脱困策略将最长卡死时间从十余秒压缩至约 2 秒，六车场景稳定完赛，零取消资格、零严重碰撞。

1 引言

本文编写了一个自动驾驶控制函数，让仿真赛车在 Webots 环境中稳定且快速地完成规定圈数。平台每帧调用控制函数

```
control(left_img, right_img, timestamp)
```

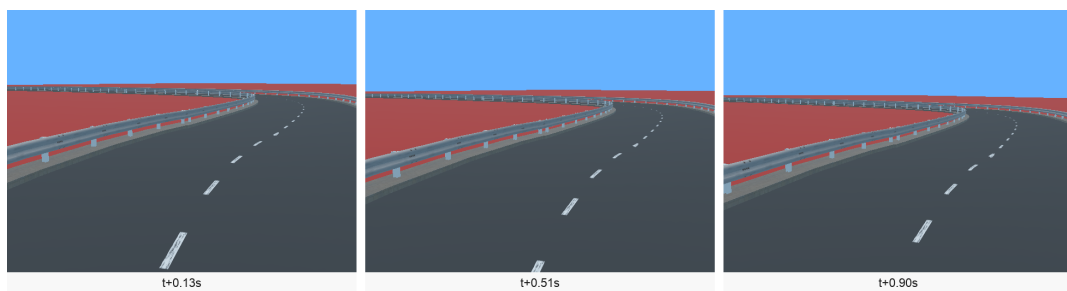
其中 left_img 和 right_img 为左右相机 BGR 图像（各 640×480 ），timestamp 为仿真时间戳，返回转向指令 $\text{steering} \in [-1, 1]$ 与速度指令 $\text{speed} \in [0, 1]$ 。算法无法读取赛道地图、车辆坐标、朝向、检查点或其他车辆位置，主要依赖摄像头画面判断道路走向与行驶策略。

图 1 展示了同一弯道处的左右摄像头输入。两路画面看到的是同一段道路，视角差异很小；控制器把它们当成两路独立观测，分别提取道路中心、白线位置和可行驶区域，再在估计阶段融合。

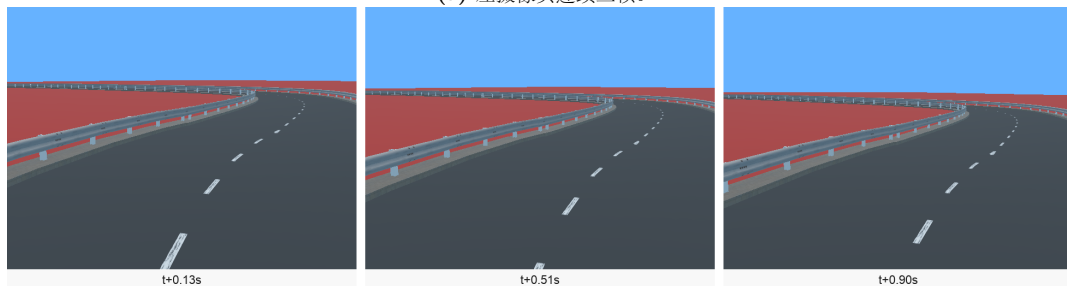
困难在于控制器的全部外部信息仅来自视觉输入，没有车速反馈，也没有惯性测量。车辆不知道自己当前是否在移动、速度多快、是否被障碍物卡住，所有状态只能从相邻帧画面的变化中推断。控制器每帧必须在数十毫秒内完成决策，无法依赖离线计算或重规划。

我们的思路是构建一条单向四级流水线，从图像中提取道路特征，将特征转换为几何状态，再根据几何状态决策转向与速度。单车计时赛主要看完赛稳定性和总用时；多车竞速赛还要控制碰撞和取消资格风险，同时争取名次积分。由于并未参加实际多车比赛（不掌握完全独立于我们自己算法的其他车的算法），后者只实现了避让和脱困算法。

单车计时赛中，控制器在 complex 赛道完整跑完一圈，总用时 252.863 秒，全程没有丢线或硬碰撞；指令平均速度达到 0.85，满速指令占 47.5%，低速爬行指令仅占 0.3%。多车场景中，早避让策略将本车陷入对手车堆的帧占比从 18.5% 降至 0.7%，丢线率从 13% 降至 0；分层脱困策略将最长卡死时间从十余秒压缩到约 2 秒，使六车场景能够稳定完赛，且没有取消资格或严重碰撞。



(a) 左摄像头连续三帧。



(b) 右摄像头连续三帧，画面与左摄像头高度接近。

图 1: 弯道处双目摄像头输入。车辆由 `webots_jump_run.sh` 放置到 `complex` 赛道历史轨迹 $t=226.56\text{ s}$ 的右弯处，图中从左到右为跳点后 0.13 s 、 0.51 s 、 0.90 s ；该场景下双目视差不明显，后续算法仍按两路独立图像分别处理。

本文第 2 节给出控制器的整体设计与实现，包括系统架构、控制算法与程序实现；第 3 节说明我们在官方平台之外开发的回测与诊断工具；第 4 节给出单车与多车场景的实验结果；第 5 节概述开发阶段，并复盘转弯半径太小问题的定位与修复；第 6 节总结不足与经验教训。

2 控制器设计与实现

本节说明控制器如何把一帧图像变成转向与速度。

2.1 系统架构

控制器按一帧图像的处理顺序组织成一条单向流水线。每帧输入先进入感知模块 (`perception.py`)，由它从左右图像中提取道路和白线信息；估计模块 (`estimator.py`) 把这些离散观测转换成横向偏移、道路朝向、曲率等几何状态；策略模块 (`policy.py`) 据此计算转向和速度；最后由限幅函数把输出收进平台要求的范围。整体流程见图 2。

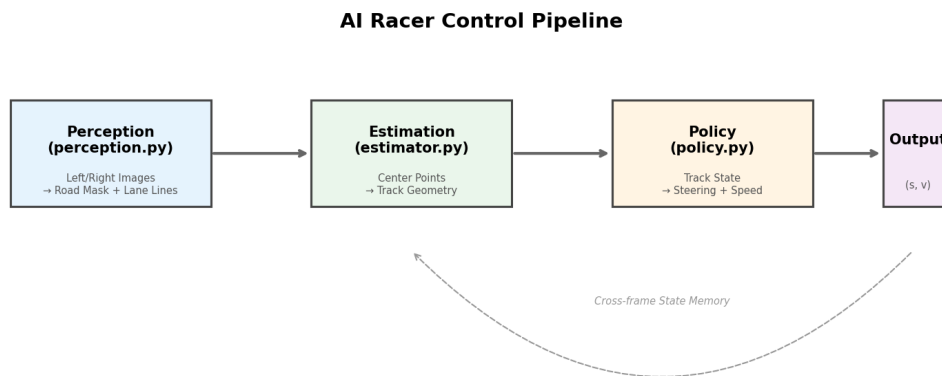


图 2：控制器四级流水线。图像先进入感知模块，再经过几何估计、控制策略和最终限幅；虚线表示跨帧状态记忆。

单向流水线意味着 `perception` 不计算控制量、`policy` 不回读原始图像、`team_controller_local.py` 只负责串接各模块。这种设计便于定位问题。看错路查 `perception`，几何量异常查 `estimator`，舵角或速度不合理查 `policy`。各文件职责见表 1。

表 1：主要文件分工

文件	负责	不负责
<code>perception.py</code>	从图像中提取道路、白线和近处对手信息	计算转向或速度
<code>estimator.py</code>	将感知结果转换为几何状态	处理原始图像
<code>policy.py</code>	根据几何状态计算控制指令	直接读取图像
<code>params.py</code>	存放两套比赛参数	执行控制逻辑
<code>opponent.py</code>	检测近处对手车身	道路分割或驾驶决策
<code>team_controller_local.py</code>	调用各模块、异常兜底、最终限幅	实现具体算法

流水线之外保留少量跨帧状态以维持动作连贯。入弯门控记住前一帧是否已入弯；脱困逻辑记录卡死持续时长；转向与速度平滑参考上一帧输出。这些状态只用于平滑动作，不改变数据单向流动的结构；各自的用途在后续算法章节展开。

模块之间通过三个数据结构交换信息，均定义在 `common.py` 中，字段见表 2。`PerceptionObs` 是感知层的统一输出，包含逐行道路中心点与左右边界点、道路宽度与置信度、白线的偏移/走向/置信度，以及仅在多车 `profile` 下有效的对手标志、横向位置、尺度和画面流动量。`TrackState` 是几何估计的输出，也是策略模块的唯一输入，它将近处横偏、远处预瞄横偏、走向、曲率、置信度、丢线标志等几何量，连同白线、边界余量、对手等透传字段一并交给策略，使策略无需回读原图。`ControlCmd` 仅含 `steering` 和 `speed` 两个标量。各字段的具体含义在下文算法中随用随解释，数据结构在构建与测试中的强制约束见第 2.3 节。

表 2: 模块间的三个数据契约 (common.py)

数据结构	主要字段	含义
PerceptionObs (感知输出)	center_points / left_edge_points / right_edge_points	逐行道路中心点与左右边界点
	road_width_est, confidence	道路宽度估计、感知置信度
	line_offset / line_heading / line_confidence	白线偏移 / 走向 / 置信度
	near_obstacle / obstacle_x / obstacle_size / frame_motion	近处对手标志 / 横向位置 / 尺度 / 画面流动量
TrackState (几何状态)	lateral_error / lookahead_error	近处横偏 / 远处预瞄横偏
	heading_error / curvature	道路走向 / 曲率
	confidence / lost / red_environment	置信度 / 丢线标志 / 红色场地标志 (其余为感知透传)
ControlCmd (控制命令)	steering / speed	转向 / 速度指令

2.2 控制算法

以下按数据流顺序介绍算法。感知模块在无地图条件下从双目图像中提取道路几何，估计模块将离散观测点转换为连续状态量，策略模块根据几何状态计算转向与速度。多车场景会在这条主链路上再叠加对手感知、避让与脱困；单车和多车行为通过 profile 机制隔离。

2.2.1 感知 (perception.py)

感知模块在没有先验地图的条件下，从单帧 BGR 图像中识别可行驶区域和车道中心线。处理流程依次为颜色分割、扫描线追踪和白线检测。

第一步是颜色分割。我们从真实帧中采样六类颜色，分别对应深灰沥青路面、较亮路肩、蓝灰色护栏、绿色草地、红色场地和蓝色天空；每类在 HSV 和 Lab 两个色彩空间中各存一组中位数和容差，作为色卡。逐像素比对后，符合路面颜色特征的像素会被标成道路区域。这一步得到道路掩膜。掩膜是一张二值图，白色像素表示路面，黑色像素表示非路面。随后从道路掩膜中扣除草地、红地、天空和偏亮路肩等误检区域。横跨赛道的蓝色检查点门虽然颜色特殊，但门后仍是路面，因此把门所在的整行像素也并入道路掩膜。蓝色门与蓝色天空颜色接近，天空一旦被误纳入道路掩膜，后续扫描线就会把天空当成可行驶区域。为此我们先用形态学开闭运算去掉零散误检并填补道路内部的小空洞，再通过连通域过滤只保留面积足够大且位置合理的连续区域。同时把图像边缘单独保留成一份备用掩膜，仅在道路掩膜完全失效时作为兜底参考。

第二步是扫描线追踪。我们在画面中下部的感兴趣区域内布置多条水平扫描线（当前为 12 条），从近到远逐行扫描。在道路区域内，每行取连续路面区间的中点作为该行道路中心的候选点；如果相邻行之间的中点位置突然变化过大，就丢弃该点。当道路在远处分叉，或被对手车辆遮挡而出现多个候选段时，系统用上一帧保存的道路中心位置作为参照，选择与上一帧最接近的候选。这样可以保持追踪连续。complex 赛道 CP3 区域有多段道路在画面中汇合，这个时间锚点尤其重要。输出包括左右相机各自的逐行道路中心点序列，以及对应的左右边界点。

第三步是白线检测。白色虚线标出车道中心，通常比道路区域的中点更可靠。弯道内侧的草地往往比外侧窄，道路区域的中点会偏向内侧，而白线不会。白线的判定需同时满足三个条件。足够亮、色度接近中性白（通道间的最大差值小于设定的阈值，护栏因偏蓝灰色度更高而被排除）、左右紧邻都是深灰色路面（排除偏蓝灰的护栏支柱和外侧路牙）。检测后输出白线相对车中轴的横向偏移（正为偏右）、白线的走向以及置信度（按检测到的白线点数计算）。图 3 展示了一个急左弯的感知结

果。青色横线是扫描线，黄色框是白线候选，红点是每行道路中心。急弯中仍能得到连续的中心点和白线点。左右两目的检测在感知阶段独立完成，估计模块再按白线置信度和道路点数量融合结果。

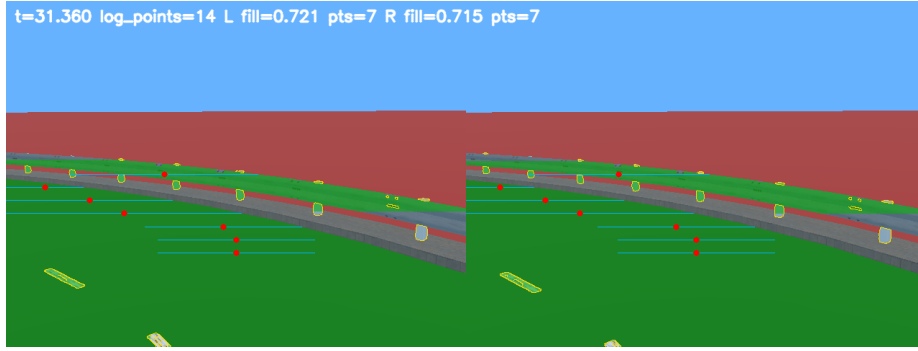


图 3: R049 第一个急左弯中段的双目感知叠加图。左右两幅为左右相机，红点是逐行道路中心，黄框是白线候选；急弯里仍有连续的中心点和白线点支撑转向。

2.2.2 估计 (estimator.py)

估计模块把感知输出的离散道路点转换成四个连续量，供策略模块使用。近处横偏表示车辆相对于近处道路中心的位置（正为偏右，负为偏左）；远处预瞄横偏表示前方较远处的道路中心位置；道路走向表示当前道路的方向；曲率表示弯道的缓急程度。计算时先把图像坐标归一化，再对道路中心点做多项式拟合。点数足够时用二次曲线拟合，点数不足时退化为直线。近处横偏取近端点的中位数，远处预瞄横偏取远端点的中位数，道路走向取拟合曲线在中间位置的导数，曲率取拟合曲线的二次项系数。当白线可信时，横偏、走向和预瞄会按置信度向白线方向靠拢。白线通常比道路区域的中点更可靠，弯道中尤其如此；但白线横偏过大时不予采纳，因为这种检测结果往往来自路沿或远处干扰线。

曲率可信度用于压制不可靠的二次项估计。当道路点数太少、纵向覆盖范围不足或拟合误差较大时，拟合出的二次项多半是噪声，不代表实际弯道曲率。此时我们把曲率向零平滑，避免远处少量稀疏点制造出虚假的急弯。入弯时机的判断只用近处横偏，原因在下一节展开。

2.2.3 策略 (policy.py)

策略模块的输入是估计模块给出的几何状态，输出是转向和速度两个标量。它先从几何状态中算出若干风险分量，每个分量都落在 0 到 1 之间，用来衡量某一类危险。曲率风险取曲率、走向、远处预瞄横偏三者绝对值的最大值（弯越急或偏得越远，该值越接近 1），横偏风险取近处横偏的绝对值，此外还有置信风险（即 1 减置信度）、丢线风险（丢线时为 1、否则为 0）和边界余量风险。曲率、横偏、置信、丢线四项加权求和后得到综合风险值，供速度和模式选择共同使用。模式由一个五态状态机决定，在巡航、入弯、修正、恢复、丢线之间切换。巡航对应直道，允许满速；入弯对应高曲率，允许大舵角；修正对应中等横偏；恢复是丢线后的缓冲；丢线状态则按上一帧的方向保守滑行。状态机使用迟滞，进入和退出同一个状态采用不同阈值，避免在临界点附近反复跳变。

目标舵角由两项构成。近处回中项按近处横偏乘一个增益，把车拉回当前道路中心；远处预瞄项由远处预瞄横偏、走向和曲率三者加权，让车提前为前方弯道做准备。两项再接近、远权重合成最终舵角。

$$\underbrace{c = K_{\text{lat}} e_{\text{lat}}}_{\text{近处回中}}, \quad \underbrace{\ell = K_{\text{look}} e_{\text{look}} + K_{\text{head}} e_{\text{head}} + K_{\text{curv}} \kappa}_{\text{远处预瞄}}, \quad s = w_{\text{near}} c + w_{\text{far}} (g \cdot \ell)$$

其中 e_{lat} 、 e_{look} 、 e_{head} 、 κ 依次是几何状态里的近处横偏、远处预瞄横偏、走向和曲率，都由估计模块算出；四个增益 K_{lat} 、 K_{look} 、 K_{head} 、 K_{curv} 是调定的常量，存放在共享参数表中。近处权重 w_{near} 与远处权重 w_{far} 会在基准值上叠加随风险变化的增量。横偏风险越大，近处权重越高，控制器越偏重把车拉回中线；曲率风险越大，远处权重越高，控制器越偏重提前为弯道转向。 $g \in [0, 1]$ 是入弯门控乘子。车还没到弯口时， g 接近 0，把整个远处项压住，避免直道上提前转向。这个门控机制在下一节展开。

入弯门控直接决定转弯半径。问题在于远处预瞄项在直道后段看到前方弯道时就会起作用，提前向弯内侧打方向，造成切内线。门控用一个连续乘子控制远处预瞄项的释放时机，让车先沿直道开到弯口，等横偏出现后再逐步加大转向。这个乘子 g 叫 `corner_arrival`（入弯到达度），只由近处横偏的绝对值决定。

$$g = \text{clamp}\left(\frac{|e_{\text{lat}}|}{r_{\text{turn}}(1 - \beta v_{\text{norm}})}, 0, 1\right), \quad v_{\text{norm}} = \text{归一化车速}$$

直道上车居中， e_{lat} 接近零， g 也接近零，远处预瞄项被压住；车到弯口后向外漂， e_{lat} 变大， g 随之升高，逐渐放出预瞄项。式中 r_{turn} （参数名 `turn_in_lateral_ref`，取 0.75）是门控的归一化基准，调小它会让门更早打开、入弯更早、半径更小； β （`turn_in_speed_comp`）和 λ （`turn_in_hold_decay`）也都是调定的常量，作用见下文三点。 v_{norm} 是归一化车速。平台不提供车速反馈，我们用上一帧实际输出的速度指令除以最大速度来近似当前车速。

这套门控有三点值得展开。第一，判据只用近处横偏，不用走向或远处曲率；后两者在直道接近段就可能偏大，用作判据会提前开门。入弯瞬间，远景弯道尚未充分进入视野，也无法可靠判断弯的缓急。第二，门控与速度耦合。车速越高，横偏长起来之前已经多跑了一段距离，因此高速时按 β （当前 0.6）收小有效基准，让门提前开。第三，门控带保持与出弯迟滞。横偏在弯中会短暂回落，若不保持，远处预瞄项会被收掉，车容易转到一半就收轮；进弯后把乘子锁到峰值，出弯时按 λ （当前 0.92）逐步松开。第 5 节的典型案例给出这些设计的演进证据。

门控之外还有一道白线减预瞄修正。当可信白线显示车已经骑到弯内侧（白线横偏 `line_offset` 与远处项符号相反）时，控制器按内侧程度和白线置信度成比例削弱远处预瞄项，并用迟滞保持这次修正，打断切内、往外打、过头后再切内的来回振荡。这个修正只缩远处转向项，不动风险和速度。合成出的舵角输出前还要经过三道平滑。第一道按模式做指数平滑，新值与上一帧加权混合，越接近巡航越平滑；第二道限制每帧变化幅度，避免舵角突跳；第三道在高速时收小最大舵角。速度从基准速度 v_{base} 出发，连乘弯道、横偏、置信、转向四个降速因子，任一方面风险升高都会压低速度。

$$v = v_{\text{base}} \cdot \underbrace{(1 - \eta_c R_{\text{curve}}^p)}_{\text{弯道}} \cdot \underbrace{(1 - \eta_o R_{\text{off}}^q)}_{\text{横偏}} \cdot \underbrace{f_{\text{conf}}}_{\text{置信}} \cdot \underbrace{(1 - \eta_s |s|^r)}_{\text{转向}}$$

其中 R_{curve} 、 R_{off} 是本节开头算出的曲率风险和横偏风险， $|s|$ 是上面合成的舵角绝对值； v_{base} 是基准速度（取 0.96）。每个因子中的 η 是该项的降速强度，指数 (p, q, r) 控制风险到降速的映射曲线，都是调定的常量。例如弯道因子取 $\eta_c=0.42$ 、 $p=1.5$ ，只压急弯而放过中等弯（效果见第 5 节的典型案例）。置信因子 $f_{\text{conf}} = \text{min_confidence_factor} + (1 - \text{min_confidence_factor}) \cdot \text{confidence}$ 把置信度线性映射到一个不低于 0.95 的系数，使中等置信几乎不再无谓压速。单帧加速幅度设了上限，避免丢线恢复后速度突然窜出；还有一道后置白线修正，在所有计算完成后把舵角朝可信白线方向轻推一点，同样只改舵角、不进风险、速度或门控。

表 3 汇总上述感知与控制的主要参数。这些驾驶参数由两个 profile 共享。单车与多车使用同一辆车，物理条件一致，因此取值相同；多车专属的避让与脱困增量见第 2.2.4 节表 4。

表 3: 主要驾驶参数总览（两 profile 共享，取值相同；详细含义见正文与各算式）

阶段	符号 / 参数名	值	一句话作用
感知	scan_count	12	扫描行数；弯中虚线靠加密行召回
	white_chroma_max	40	白线允许的最大通道色度，排除蓝灰护栏
	offset_trust_max	0.75	白线横偏信任门，超出即拒绝
入弯门控	$r_{\text{turn}} / \text{turn_in_lateral_ref}$	0.75	门控归一化基准（小 = 早转、半径小）
	$\beta / \text{turn_in_speed_comp}$	0.6	速度耦合系数，高速提前开门
	$\lambda / \text{turn_in_hold_decay}$	0.92	弯中峰值锁存衰减，防转一半收轮
转向	$K_{\text{lat}}/K_{\text{look}}/K_{\text{head}}/K_{\text{curv}}$	0.80 / 0.68 / 0.68 / 0.12	近处回中与远处预瞄三分量的增益
	max_abs_steering	0.76	最终舵角绝对值上限
速度	$v_{\text{base}} / \text{base_speed}$	0.96	基准速度
	$\eta_c / \text{curve_slowdown}, p / \text{curve_power}$	0.42, 1.5	弯道降速强度与曲率指数（只抬中等弯）
	min_confidence_factor	0.95	置信降速下限，解耦中等置信的无谓压速

2.2.4 多车层

多车层只在 with_other_cars profile 中启用。它在单车驾驶底座上叠加对手感知、避让和脱困三类能力，由 enable_opponent 总开关控制。

感知端在画面下半部中间的检测区内搜索三类车身候选，包括亮白区域、近黑区域和高饱和的官方彩色车身。低饱和色卡会被排除，因为容差太宽会把 complex 赛道的深灰沥青误判为车身。候选区域经过去噪和连通域过滤后取最大块，输出近处对手标志、对手横向位置（左负右正）和对手块尺度（越大通常越近）。画面流动量取相邻帧下采样灰度图的平均绝对差，用于判断画面是否还在移动。实测标定显示，正常行驶时该值不低于 0.47、中位约 0.81，被顶住空转时约 0.07，差异足够触发卡死判断。

避让分两步。第一步是减速。检测到近处对手时，目标速度按对手居中程度在正前因子（0.72）和偏侧因子（0.92）之间插值，计算式为 $0.92 + (0.72 - 0.92) \times \text{居中程度}$ 。对手越居中，减速越多；对手偏在一侧时只轻微减速，避免一被超车就长期跟在后面。如果同时正在弯中，再额外乘一个弯道对手减速因子（0.55），防止多车在弯里相撞卡死。第二步是有界转向偏置。控制器朝边界余量更大的开阔一侧加偏置绕行，同时朝对手的反方向让开。

$$b = \text{clamp}\left(\underbrace{g_m \Delta_{\text{margin}}}_{\text{朝开阔侧}} - \underbrace{g_d x_{\text{obs}}}_{\text{朝对手反侧}}, -b_{\text{max}}, +b_{\text{max}}\right), \quad s \leftarrow s + b$$

其中 Δ_{margin} 是左右边界余量差， x_{obs} 是对手横向位置（左负右正）， $g_m=0.65$ 、 $g_d=0.20$ 是两个增益，偏置整体钳在 ± 0.42 以内。这个偏置只改最终舵角，不进入风险计算和速度决策。

脱困逻辑分为四层（图 4）。上层针对更具体、更危险的楔住情形，下层负责兜底；命中靠上的层后不再继续判断。四层都有两个共同要求。触发条件必须连续保持若干帧，相邻帧画面也要足够稳定，避免一帧噪声触发；动作大多先倒车拉开再前冲，并统一朝边界余量更大的开阔一侧脱困。

第一层，顶栏脱困。这一层处理车身斜着楔进栏杆、几何状态几乎不动的情形。触发时近处横偏很大，舵角已经打到很大，车速却很低；也就是车顶住了，但速度还没低到下一层的阈值。动作是朝远离栏杆的一侧打大反向舵角，先倒一段拉开距离再前冲，相当于一次三点掉头。

第二层，边界障碍脱困。这一层处理一侧被静态物顶住的情形，典型例子是停住的对手车。触发时系统已经检测到近处障碍，车辆正在转弯或修正，某一侧边界余量接近零，车速很低，但仍看得见路。动作是朝余量更大的开阔一侧打舵，低速挪出。

第三层，低速失速脱困。这一层处理指令速度持续接近零、车辆几乎停住的情形。判据是指令速度低于阈值并持续若干帧。动作是让前轮按方波左右摆动，在约 0.95 的基础脱困舵角上叠加正负 0.30、每十帧翻向一次（见图下半），靠反复摆动打破静摩擦、让轮胎重新咬地。

第四层，强制脱困安全网。前三层都没接住时，再由这一层兜底。持续丢线超过 45 帧、指令速度低于 0.08 持续 60 帧、或画面流动量（frame_motion）低于 0.28 且指令速度不低于 0.40 持续 30 帧，任一条件成立都会触发。第三项是光流卡死检测，用来在没有车速反馈时识别撞栏顶停。此时指令以为车辆在巡航，画面却几乎不动。动作是先朝开阔侧净倒一段，把车从车堆里整体倒出来，再前冲脱离。

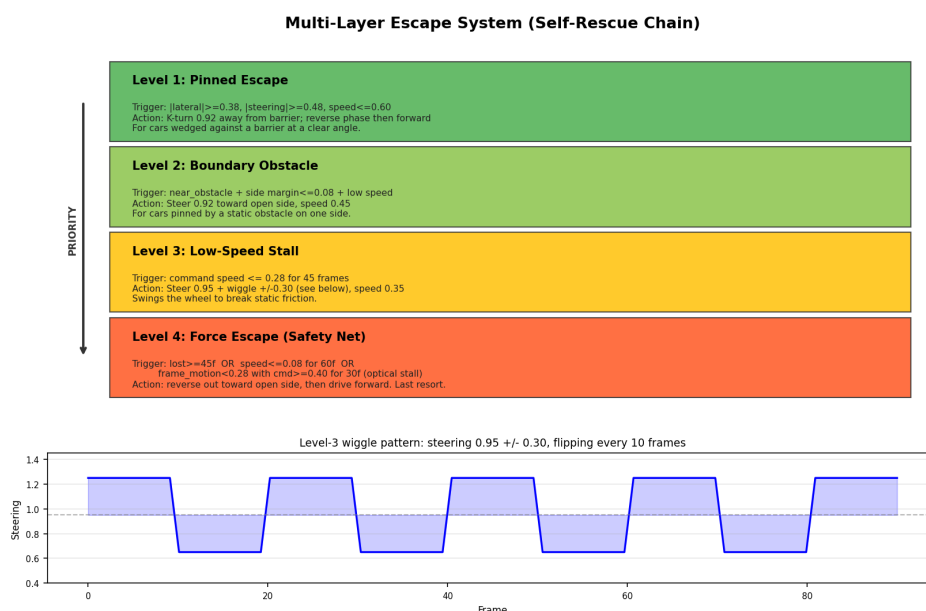


图 4：四层脱困自救链按优先级排列。上方层级先评估，每层标注触发条件与动作；下方子图示意第三层的方波摆动，转向在 0.95 上下摆动正负 0.30、每十帧翻转一次。

脱困逻辑中包含倒车动作，但倒车在本地和线上环境的规则不同。在本地 Webots 仿真中，控制器输出负速度会让车辆后退；线上评测平台会将速度指令截断到 $[0, 1]$ ，负速度退化为原地停车。因此每一层脱困都预留了前进相位作为兜底。倒车在本地有效，在线上虽然失效但不会造成危害，车辆随后仍会按前进相位尝试脱困。

表 4 汇总多车专属参数，只在 with_other_cars 中启用；no_other_cars 把这些键全部覆盖为禁用值（因子取 1.0、增益与摆幅取 0、阈值取 0 使其永不触发），让单车彻底退化为纯 R049 驾驶底座，机制见第 2.2.5 节。

2.2.5 Profile 隔离

Profile 隔离用于确保单车和多车两种场景下的行为完全独立。接口本身不区分场景，因此我们在参数和运行路径两处做隔离。

表 4: 多车专属参数（仅 with_other_cars 启用；no_other_cars 全部禁用）

分类	参数名	with_other_cars	no_other_cars	一句话作用
避让	enable_opponent	True	False	多车能力总开关
	opponent_speed_factor / _side	0.72 / 0.92	1.0 / 1.0	正前 / 偏侧对手让速因子
	opponent_corner_speed_factor	0.55	1.0	弯中遇对手额外减速
	opponent_avoid_steering_max	0.42	0.0	绕行偏置 $b_{\max}$ 上限
脱困	escape_wiggle_amplitude	0.30	0.0	低速失速时方波摆幅
	motion_still_threshold	0.28	0（禁用）	光流卡死阈值

参数上，单车参数字典从多车字典派生，共享入弯门控、速度、曲率等主要驾驶参数，但会覆盖两类键。脱困参数改用更保守的值，所有多车增量参数（包括对手检测开关）置为禁用。

运行时，当前 profile 贯穿整条流水线。perception 每帧根据开关决定是否运行对手检测和画面流动计算；单车下两者都不运行，近处对手恒为假，对手位置和尺度恒为零，画面流动量保持默认高值，卡死检测永不触发。policy 中的强制脱困和卡死触发也用同一个开关再守一道。单车直接跳过多车代码路径，同时每帧省下约 5 毫秒的检测开销。这套约束由 test_profile_isolation.py 测试用例锁定。

2.3 程序实现

代码集中在 controller/ 目录，每个文件按数据流顺序承担单一职责。common.py 在最底层，定义三个 dataclass 契约和 clamp_cmd 限幅函数。params.py 集中存放全部可调参数，按场景派生出 no_other_cars 与 with_other_cars 两套常量字典，由 get_profile 按名分派，任何模块不得在本地硬编码参数值。

感知入口是 extract_observation._build_masks 做 HSV 与 Lab 双色彩空间的色卡比对并生成道路掩膜；_scan_image 沿扫描线逐行取中点，遇分叉或遮挡时用上一帧道路中心做时间锚点匹配；_camera_line_state 做白线三重判定；_fuse_scans 把左右两目结果按置信度加权融合。对手检测放在独立的 opponent.py 中，入口 detect_near_vehicle_obstacle_state 只在多车 profile 下调用。

估计入口是 estimate_track，策略入口是 decide_control。策略层依次计算风险、选择模式、生成舵角、生成速度，并在最后处理四层脱困链。team_controller_local.py 不含算法，只负责按 profile 调用感知和策略、异常兜底（try/except 返回零速直行）以及调用 clamp_cmd 限幅。

三个数据结构是模块间的硬契约，用 dataclass 实现，字段固定、不包含方法。改动任一字段必须同步更新全部调用方、构建脚本和测试，否则拼接出的单文件会因字段不匹配在运行时出错；专门的测试会锁定字段集合。限幅由入口层的 clamp_cmd 统一执行。steering 钳在 $[-1, 1]$ ，speed 下界放宽到 -1.0 以支持本地倒车脱困（线上沙箱把 speed 钳回 $[0, 1]$ ，倒车退化为原地摆轮）；构建脚本自动把 clamp_cmd 拼到单文件末尾，确保限幅在所有算法之后执行。

提交文件由 build_submission.py 生成，按 common $\rightarrow$ params $\rightarrow$ opponent $\rightarrow$ perception $\rightarrow$ estimator $\rightarrow$ policy $\rightarrow$ team_controller_local 的固定顺序拼接各模块。这个顺序由依赖关系决定。common 被所有模块引用，必须排最前；params 提供各模块的常量字典；opponent 独立于感知主链路，排在 estimator 前即可；policy 依赖 TrackState，故排在 estimator 后；team_controller_local 引用所有上游模块，排最后。拼接时脚本删除跨文件 import、剔除所有调试 I/O（open、json.dump、cv2.imwrite 等），并注入 team_controller_local.PROFILE 常量决定运行时使用哪套 profile。产物不依赖 controller 目录外的文件，可在任意 Python 3.9 以上环境独立运行。带调试输出的版本用 -debug-log 和 -dump-frames 构建，仅供本地分析，绝不上传；第 3 节的回测取证依赖这些本地调

试产物。

校验分两层。内层 `validate_submission.py` 直接 import 生成的 `team_controller.py`，用 mock 图像多次调用 `control()`，检查函数签名、返回类型、`steering` 与 `speed` 的值域，以及 import 是否清除、是否残留禁用模块和调试 I/O。外层是官方 `validate_controller.py`，在沙箱中逐行扫描是否使用了 `os`、`sys`、`time`、`threading`、`open`、`eval` 等禁用模块与内置函数，并核对单帧耗时在预算内。仓库另有 149 个 `pytest` 用例，覆盖接口契约与值域 (`test_contracts`)、几何拟合正确性 (`test_estimator`，含直道曲率近零、弯道曲率符号与走向一致)、舵角与速度的边界 (`test_policy`)、车身检测的假阳性排除 (`test_opponent`)，以及单车下对手字段恒为默认值的隔离约束 (`test_profile_isolation`)。感知与脱困的正确性，由这些用例配合第 3 节的 Webots 实跑共同保证。

3 回测与诊断系统

控制器在比赛中几乎无法获得外部数据。它不知道自己的真实坐标和速度，也不知道是否撞了护栏。平台的 `telemetry` 只记录车车碰撞，撞护栏和静态几何体一概不报。离线统计量本身也会误导判断。以丢线率为例。有一版因蓝色场景接近天空导致道路掩膜饱和，被判成丢线，但车实际在平稳滑行；另一版把草地当路面，判为正常却朝草地开。实跑又带有随机性，同一版代码两次跑的轨迹也不会完全一致。只靠改一版、上传、看结果很慢，也说不清改动到底改善了哪里。为此我们围绕本地 Webots 搭建了一套回测与诊断系统，用本地实跑、逐帧留档、接触检测和量化脚本支撑迭代。它可以反复执行，大部分流程也能无人值守，便于 AI 自行迭代优化，而不必每一步都由人工介入。

这套系统的代码集中在 `scripts/` 目录，与第 2.3 节的构建/校验工具同处一处，但职责不同。`webots_run.sh`、`webots_jump_run.sh`、`webots_auto_multicar.sh` 等 shell 脚本调用官方 SDK 的 `run_local.py` 启动 Webots，负责产生实跑；`analyze_*.py`、`plot_*.py`、`replay_offline.py` 等 Python 脚本消费实跑产物，负责判读。每帧控制日志、相机帧、接触日志、遥测复制件由调试版控制器和带开关的 `run` 脚本写到 `.tmp/` 下，从不上传，并按轮滚动归档；需要进报告或长期回查的精选图，再提升到 `experiments/figures/`。唯一不在 `scripts/` 里的部件是撞栏接触检测，它注入在官方 SDK 的 `supervisor` 进程里，详见下文。

3.1 整场实跑与逐帧留档 (`webots_run.sh`)

该脚本负责自动记录整场实跑的每一帧，便于事后逐帧回看，避免为查看某一时刻而重跑两百多秒。`webots_run.sh` 将流程整合为一键操作。它会用调试模式构建控制器、清理残留 Webots 进程、调用 `run_local.py` 跑完整场 `basic` 或 `complex` 赛道，并在跑完后归档。默认每十帧（约 0.3 秒）保存一对无损左右相机 PNG。这个间隔足够支撑整场回看，又不至于使产物膨胀到数 GB；如需精确定位某次撞栏，可单独以逐帧（stride 1）重存那一小段。控制器每帧的内部状态（模式、目标与最终舵角/速度、白线置信度等）由调试构建写入 JSONL 格式的控制日志，一行一帧；撞栏接触日志也默认开启。

为防止取证帧被覆盖，每场开始前脚本会将上一轮的 `.tmp` 产物和 SDK 遥测滚动归档到 `.tmp/run.archive/`，保留最近十轮。早期曾因撞栏帧被提前清掉而被迫重跑取证，这个机制正是为此而加。多车场景另有一个无人值守版本 `webots_auto_multicar.sh`。它在后台启动六车 Webots 并挂载一个看门狗，超时或日志长时间不动即自动收尾；车数、最大时长、静止判定秒数和对手速度缩放均可配置。将对手速度缩放设为小于 1 的值，即可得到一个对手统一变慢的纯超车测试场景，用于单独检验本车能否干净地追上并超过慢车。有了这套工具，改代码、自动跑、读日志的循环可以整夜无人值守地执行。

3.2 跳点续跑与确定性摆位 (webots_jump_run.sh、make_teleport_world.py)

两件事拖慢了迭代。第一，复现某个弯或某个卡点往往需要从发车格重跑两百多秒。第二，对手正好斜停在正前方这类多车情形，在六车混跑里几乎只能靠运气遇到。为此我们用 `webots_jump_run.sh` 配合 `make_teleport_world.py` 做断点续跑。脚本从一份历史遥测中取出某一时刻的位姿（坐标和朝向），由 `make_teleport_world.py` 临时生成一个将车直接放在该位置的 Webots world，再跑几秒并存帧。这样验证当前代码从这个姿态会怎么开只需要几秒。需要强调的是，该工具只恢复位姿，不恢复线速度、轮胎接触和控制器内部记忆，是一种近似续跑。它足以回答从这里出发大概会怎么打轮、画面里有没有白线，但不能据此断言车辆能从该状态真实脱困，那仍需要从头跑正式的 world。

`make_teleport_world.py` 还能脱离历史遥测直接指定位姿，并按链式方式摆放多辆车。配合一个每帧返回零指令的静止对手控制器，就能确定性地构造出对手贴边停或对手斜跨整条车道这类受控场景，而这些在六车混跑中几乎不可复现。脚本可以一键完成摆位、headless 实跑（`--fast --batch`，即无界面快速批处理）和离线出图，无需逐场人工盯守。后文图 7 与图 8 的两组避让取证，正是通过这种方式摆出来再跑的。

3.3 撞栏接触检测 (SDK supervisor、analyze_contact_log.py)

单车驾驶质量很大程度上取决于是否蹭栏以及蹭栏的严重程度，但平台 telemetry 只记录车车碰撞，撞护栏和静态几何体一概不报。离线分析长期只能从丢线率和横偏等指标间接推测，无法直接确认撞栏。

为此我们没有修改控制器代码，而是在官方 SDK 的 supervisor 进程中注入了一段接触检测逻辑（supervisor 掌握全局世界状态，负责计分与判定），通过环境变量开关控制，仅在本地生效。它调用 Webots 的 `getContactPoints()` 逐帧提取车身与护栏、静态几何体的接触点，输出结构化日志（时间、世界坐标、接触点数、最大穿透深度 `zmax`），同时向 telemetry 事件流中插入 `contact_start` / `contact_end`，使总览图能在时间轴上标注接触时段。

难点在于区分真实撞栏和发车格的底盘伪接触。发车时底盘本就贴着路面，会持续报告接触点。判读规则来自对大量实跑的标定。同一窗口内接触点数峰值不少于 3，且 `zmax` 超过 0.6，判为真实撞栏；只有孤立的一两点，或 `zmax` 落在发车格路面的固定值附近（约 0.49），判为伪接触并忽略。多车日志包含全部六辆车的数据，必须按车位过滤，否则会把对手的撞栏计入本车。`analyze_contact_log.py` 将接触帧聚合成一次次撞栏事件，汇总每次的时间、位置、点数和严重程度。至此，离线分析终于能直接定位每次撞栏。后文图 5 中叠在轨迹上的七处轻微接触点，就来自这条链路。

3.4 逐帧感知标注 (analyze_perception_dump.py)

走线出问题，我们需要分清控制器是看错了路（感知错误），还是看对了路却打错了轮（策略错误），这要求能还原某一帧里控制器究竟看到了什么。

`analyze_perception_dump.py` 读取已保存的相机帧，对指定时刻重新跑一遍感知流程，并把中间结果叠加回原图。道路掩膜、边缘、各条扫描线、逐行道路中心红点、白线候选黄框一并叠加，同时标注走廊填充率和道路点数。它还读取同一时刻的控制日志，因此叠加图上既能显示感知结果，也能显示当时打出的舵角，从而直接区分上述两类错误。另有 `replay_offline.py` 能把同一批已保存的相机帧重新输入给当前代码，在固定画面下做改动前后的字段级对照（消除实跑随机性后精准判断改动是否生效），个别走线改动用它确认过。

多车 profile 下，叠加图还会额外画出对手检测框和 `obstacle_x` 竖线（对手在画面中的横向位

置，左负右正)，用于核对避让时看见了哪块车身、判它在左还是在右。本文大部分感知证据都来自这条脚本。图 3 的急左弯叠加、图 16 的稀疏白线残留窗口，以及图 7 与图 8 的相机叠加，均由它生成。

3.5 整场量化判读与顶视复盘 (analyze_*.py、plot_run.py、plot_topdown.py)

这组脚本把整场实跑压缩成数字和一张顶视复盘图。

最常用的是 plot_run.py。它将整场轨迹按速度着色绘制成顶视图，配一条速度-时间曲线，并叠加事件和撞栏接触点。本文每个版本的总览图都由它生成，如后文图 5。更细的文字摘要由两支脚本给出。analyze_telemetry.py 从遥测中读取位置、速度、爬行段和事件，用来判断车跑到哪、有没有卡住；analyze_control_log.py 从控制日志中读取模式、舵角、速度、白线置信度等每帧内部状态，用来解释控制器为什么这样打轮、为什么这样减速。

再往上是两个面向具体问题的专项脚本。analyze_escape_episodes.py 统计每次脱困的持续时间、是否倒车、倒车后是否恢复、以及是否在正常驾驶中误触发。analyze_multicar_kpi.py 量化多车赛关注的指标，包括名次期望积分、完赛与取消资格、严重碰撞预算、超车净得失、开局互挤、卡死时长和速度（多车按名次积分而非圈速评价，见第 4.2 节）。

多车还有一条顶视复盘脚本 plot_topdown.py，配合 track_geometry.py 使用。前者把所选时间窗内各车按朝向画成车体序列，颜色浓度随时间从淡到浓；后者从赛道 world 文件中还原红色场地与灰色路面的几何。最终顶视图与 Webots 人工俯视一致，也与遥测同坐标系，可用于复盘避让绕行、发车互挤、卡死和超车。后文图 7 与图 8 的顶视图即出自此。

4 当前成绩

4.1 单车

单车计时全部来自 complex 赛道。这条赛道一圈约 1320 m，含连续急弯、内环分岔、红色场地和静态障碍段。表 5 列出最近几次官方计时。R068 的 252.863 s 是当前最佳；R069 与 R071 是清理性能软警告 W014 前后的同环境对照，结果完全一致（254.495 s），说明那次清理没有改变驾驶行为。

表 5: 单车计时成绩 (complex, no_other_cars)

Run	total_time (s)	best_lap (s)	严重碰撞
R067	254.495	254.560	0
R068	252.863	252.928	0
R069	254.495	254.560	0
R071	254.495	254.560	0

圈时之外，还需要看驾驶质量。丢线率用于衡量控制器是否偏离赛道。当道路掩膜丢失或几何估计置信度过低时，控制器会进入丢线状态；丢线率即丢线帧占总帧数的比例。表 6 使用 R049 的控制日志（8053 帧、257.7 s）做统计。丢线率为零，满速帧（speed ≥ 0.95）占 47.5%，低速爬行帧只有 0.3%。早期弯中掉速到零、再慢慢爬出的现象基本消失。图 5 的顶视图显示，整条赛道大部分轨迹是高速对应的暖色，只有弯心转冷；速度曲线进弯规律回落、出弯快速回升。整场只有七处轻微接触，接触点峰值不超过三、穿透深度不超过 0.50，没有硬撞。

图 6 左侧的速度直方图显示，指令速度主要集中在 0.8 以上，均值约 0.85、中位数约 0.89。右侧分桶图显示曲率风险越高，均速越低；表 7 给出同一控制日志的分桶统计，直道均速 0.95，中低

表 6: R049 单车驾驶质量 (8053 帧 / 257.7s, 来自控制日志)

指标	数值
丢线率	0.0%
指令均速 / 中位速度	0.85 / 0.893
满速率 (speed ≥ 0.95)	47.5%
高速率 (speed ≥ 0.85)	56.4%
低速爬行帧 (speed<0.45)	0.3%
模式占比 (巡航 / 入弯)	55% / 45%

曲率弯 0.91，中高曲率弯 0.82，急弯 0.66。curve_power 提到 1.5 后，直道和中等弯保留速度，接近物理极限的急弯才明显降速。

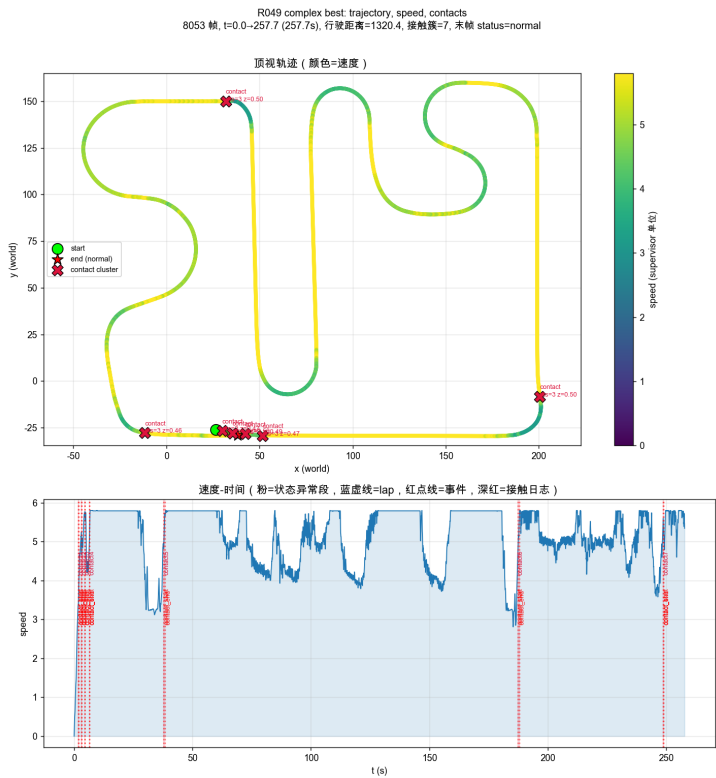


图 5: R049 整场顶视轨迹（按速度着色）与速度曲线。行驶距离 1320.4m，末态正常；暖色为高速、冷色为弯心低速，红色叉号标出七处轻微接触簇。

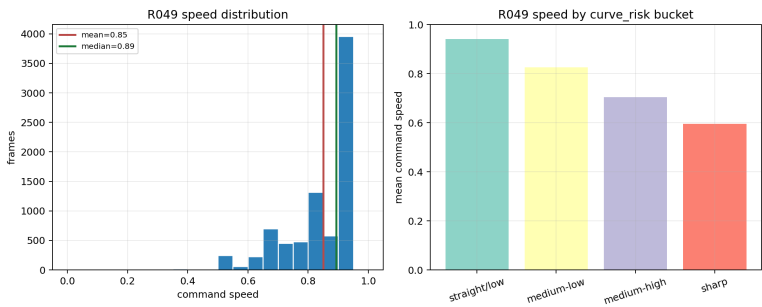


图 6: R049 指令速度直方图与按曲率风险分桶的均速。

表 7: R049 按曲率风险分桶的均速（来自控制日志）

曲率风险区间	均速	帧数占比
直道 (< 0.10)	0.950	38.3%
中低 ($0.10-0.30$)	0.912	17.6%
中高 ($0.30-0.55$)	0.816	22.4%
急弯 (> 0.55)	0.657	21.6%

4.2 多车

多车测试用同一套 `with_other_cars` 控制器同时驱动全部六个车位时，各车名次差异几乎只反映发车位置和偶发卡死，不能说明策略优劣；而且六车混跑中车与车很少贴近，难以稳定复现对手挡在正前方这一情形。为此我们改用确定性摆位做受控取证。将对手车用 `teleport` 固定到主直道上指定的位置和朝向，对手控制器每帧返回零指令使其保持静止；本车从发车格正常起步驶入，全程抽帧并记录所有车的位置和姿态。下面两幅图均来自这种设置。每幅图左侧是本车左相机的感知叠加图，右侧是顶视轨迹图。顶视图的背景（红色场地与灰色路面）由赛道 `world` 文件还原，与位姿同坐标系；车体按朝向绘制，浓度随时间从淡到浓，与 `Webots` 中人工俯视看到的一致。相机叠加图额外画出对手检测框（红框）和 `obstacle_x` 竖线（品红色，表示对手在画面中的横向位置，左负右正）。

图 7 展示了一次干净超车，对手贴着上侧护栏停放，车道留有足够通行间隙。相机叠加图中，斜前方的对手车身已被 `opponent.py` 检出（红框），`obstacle_x` 约为 -0.7 ，表示对手偏在画面左侧；`policy` 据此朝其反侧（向右，即车道下侧）施加有界偏置 b 。顶视图按时间展开本车车体。它贴着车道下侧的开阔处绕过停在上侧的对手，车体上的速度标注显示全程没有掉到零（从约 4.1 到 5.8），在 $t \approx 6.2$ 秒完成超车并随即回到满速。

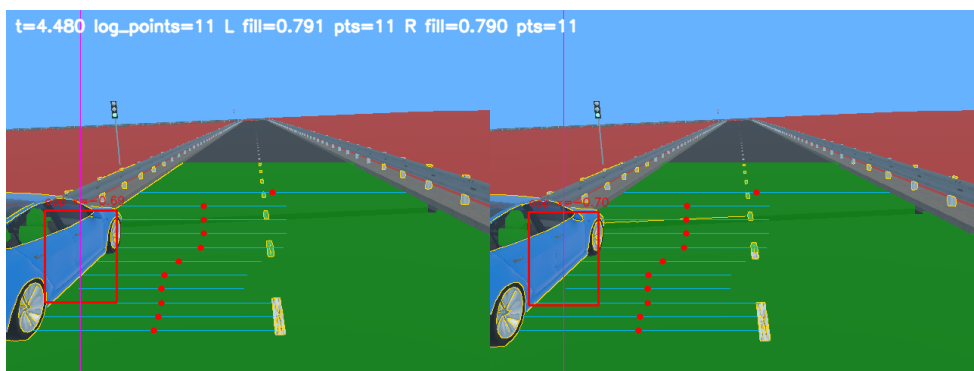
图 8 展示的是对手斜停、车身几乎横跨整条 8 米车道时的情形，也就是别的车斜着挡在正前方。相机叠加图中，斜置的对手车身占据了近半幅画面，检测框和 `obstacle_x` 同样给出。顶视图显示本车减速并朝下侧开阔处打舵尝试绕行，最后停在对手前方。控制日志也给出同样证据，目标速度约 0.65，目标转向约 $+0.66$ 。但两车各约 2 米宽，横向净空不足，本车在距对手约 3.8 米处被卡住（指令仍在前进，实际车速却接近零），随后触发 `force_escape`。图中没有展开后续脱困阶段。这组对比说明当前有界避让策略偏保守。有足够净空时能干净绕过（图 7），车道被完全堵死时则退化为脱困（第 6.1 节列为不足）。

两幅图之外，避让效果还通过一组受控测试量化，结果见表 8。第一组对照发生在弯道附近对手扎堆的窗口。无避让版本会扎进车堆，视野被对手车身遮挡，脱困状态帧占 18.5%，丢线率 13%，平均速度降至 0.67；加入早避让后，本车绕开并超过卡住的对手堆，脱困帧降至 0.7%，丢线率为零，平均速度回升至 0.85。第二组是纯超车测试。对手统一降速至 0.55，本车全速追上并超过，平均速度 0.85、中位速度 0.92、丢线率为零、全程零倒车，仅发车格有轻微接触。减速和有界绕行使本车从被夹住变为能够穿过去。

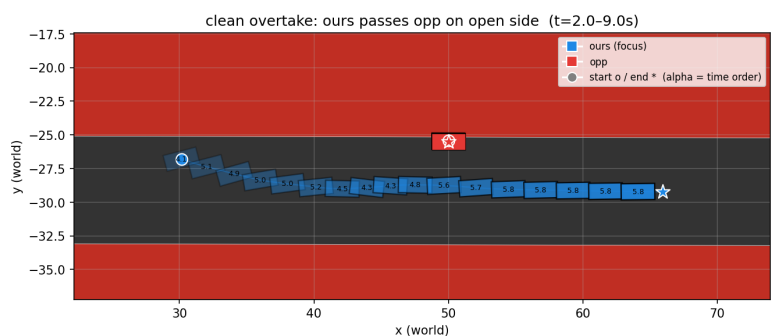
表 8: 多车避让 / 超车的受控测试效果

场景	设置	脱困帧占比	丢线率	均速	结果
穿越对手车堆	无避让	18.5%	13%	0.67	扎进车堆被糊住
穿越对手车堆	早避让	0.7%	0%	0.85	绕开并超过
追超更慢的车	对手降速 0.55	—	0%	0.85 (中位 0.92)	干净超车，零倒车

脱困层的主要作用是压缩卡死时间。图 9 对比了三轮迭代的效果。最初没有方波摆动，弯中遇对手时只会减速，卡死长达十七秒，整场失败；加入方波摆动和零速检测后，卡死时间缩短到十一



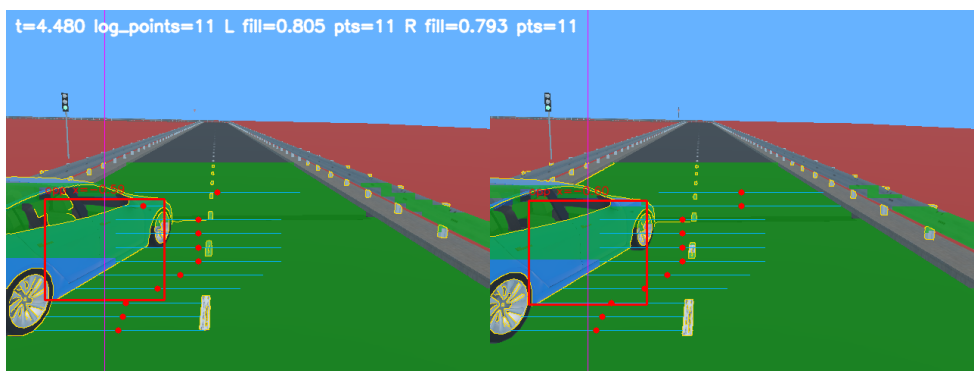
(a) 本车左相机感知叠加。对手车身被红框检出，品红竖线为 $obstacle_x$ (约 -0.7 ，偏左)。



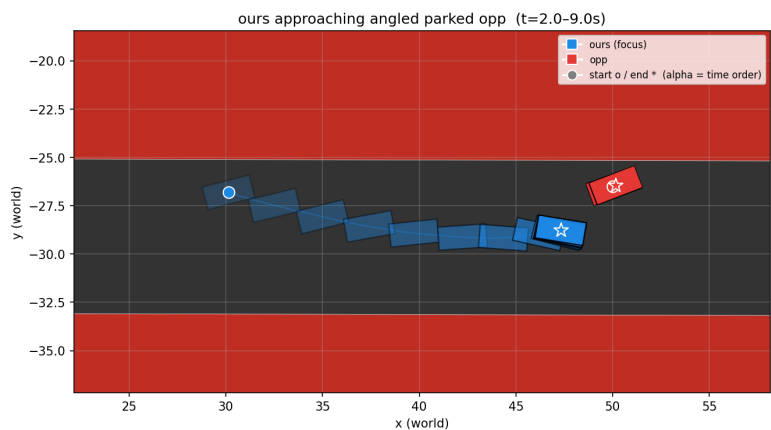
(b) 顶视图显示，本车（蓝）贴下侧开阔处绕过停在上侧的对手（红），速度标注全程不掉零。

图 7: 可通行间隙下的干净超车（确定性摆位，对手贴上侧护栏静止）。检测到对手后，控制器朝开阔侧施加有界偏置，逐步绕过对手并回到满速。

秒，但仍然无法完赛；再增加弯道减速，将摆动幅度定为 0.30 ，并提高触发灵敏度后，最长卡死时间缩短到约两秒，整场跑满两百七十多秒并完成比赛。配合光流卡死检测和倒车相位，贴边空转、车堆顶停这类刚性卡死情形也能自救恢复。六车场景最终稳定完赛，没有取消资格，也没有严重碰撞。避让减少了卷入车堆的概率，脱困机制处理了偶发的受困情况。



(a) 本车左相机画面中，斜过来的对手车身占据近半幅画面，仍被红框检出并给出 obstacle_x。



(b) 顶视图显示，本车朝下侧开阔处打舵尝试绕行，但横向净空不足，在对手前约 3.8 m 被楔住。

图 8: 车道被斜停对手堵死时的行为（确定性摆位，对手斜跨车道静止）。控制器检测到对手后减速并朝开阔侧尝试绕行，因净空不足被楔住；后续日志显示该场景会退化到脱困逻辑。

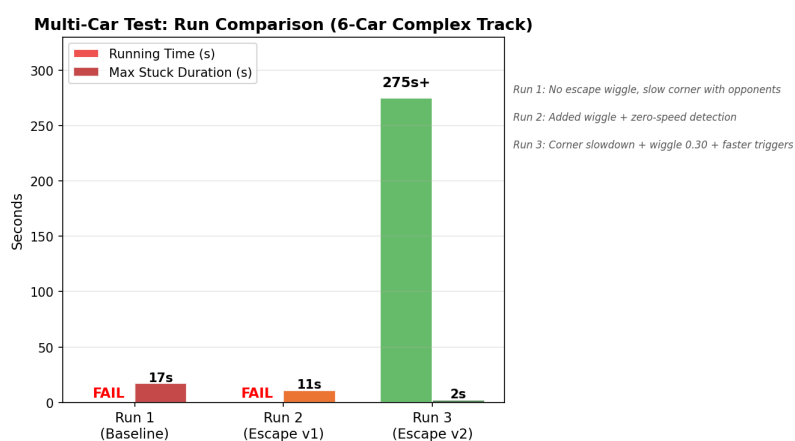


图 9: 脱困逻辑三轮迭代的卡死时间对比。前两轮整场失败、最长卡死十七秒与十一秒；第三轮跑满两百七十多秒、最长卡死约两秒。

5 探索过程

整个控制器经历了 71 次有记录的实跑（R001–R071），按目标可分为七个阶段，从 basic 椭圆赛道的基础驱动推进到 complex 赛道的单车计时与多车竞速。表 9 概览各阶段解决的主要问题。

表 9: 开发七阶段概览（R001–R071，共 71 次实跑）

阶段	run 区间	解决的问题
Stage 0	R001–010	基础驱动与感知基线，确定丢线滑行加低速脱困的兜底方案
Stage 1	R011–025	进入 complex、引入白线优先，完成 basic → complex 感知迁移；basic 椭圆稳定完赛（一圈约 288s）
Stage 2	R026–038	首轮处理转弯半径，从早切内、撞栏推进到能完整跑完一圈，但半径仍偏小
Stage 3	R039–049	系统重做入弯门控，半径与速度同步改善（本节典型案例的主线）
Stage 4	R050–052	搭起多车基础策略，验证六车能同场跑通
Stage 5	R053–061	多车脱困、避让与卡死检测；期间一次 profile 污染事故，促成单/多车参数隔离
Stage 6	R062–071	度量基建、拆分两套 profile、修好官方完赛与名次统计，并清理性能软警告

耗时最长的是 Stage 2–3（R026–R049）的转弯半径太小问题，下面把它作为典型案例展开。其余阶段的结论已分散在前文（多车避让与脱困见第 2.2.4 节、profile 隔离的机制见第 2.2.5 节）和末节的经验教训中，此处不再逐一展开。

5.1 转弯半径太小的诊断与修复

转弯半径太小的问题贯穿 R026 到 R049。急弯里半径过小会切向内线，重则擦栏，轻则在弯中把速度压到接近零、再慢慢爬出。当时还没有顶视图回放工具，无法还原车辆真实位置，因此先用图 10 说明现象。车本应沿中线行驶，到弯口附近再转向；实际却在还没到弯口时就提前打方向，以过小的半径切向内侧。图 11 用同一个第一个左弯对比早期的 R026 和后来的 R038。这两轮唯一可比的是如何通过这第一个弯，因为 R026 后来被手动停止。R026 在这个弯里速度掉到接近零、爬行十余秒；R038 仍有明显减速，但没有再掉到零，已经能通过这一弯。后续 R041 到 R049 完成诊断与修复。

分析显示，半径过小的根因是入弯太早，而入弯太早又源于早期门控用错了判据。早期门控拿道路走向和远处曲率判断车是否到达弯口，但这两个量在直道接近段就已经偏大。车还在直道上、弯口尚在前方时，远景中弯道的朝向和曲率已先进入视野，门控据此误以为车辆到达弯口，提前释放远处预瞄项，方向提前往弯内侧打，车一路切内线，半径自然偏小。图 12 用同一个最紧左弯对比旧判据和后续横偏系门控。左列是旧版走向驱动门控（R041）。入弯发生在近处横偏仅约 -0.01 、车几乎还居中时，说明门确实开得太早；此后舵角与线偏移剧烈振荡，线偏移反复越过 0.35 的内侧阈值，峰值达到 $+0.65$ ，最终十二个接触点硬撞内栏。右列是 R044 的横偏加弯急缓门控窗口，同一紧弯里舵角、横偏、线偏移几乎是一条平线，车贴着线过弯，不再撞栏。这个对照说明，先把判据从远处走向和曲率换成近处横偏，是这条主线上最重要的一步。

R044 曾尝试让门控根据弯道缓急来调节，缓弯晚开、急弯早开。结果这条分支引入了外偏，R046 便将其删除。问题同样出在入弯那一瞬间。弯道尚未在视野里完全展开，曲率风险往往偏低，于是连急弯入口都会被误判为缓弯，门控被过度延迟；等车扎进弯里、曲率风险才涨上来，门才打开，这时再急打轮，半径反而更大，车冲向外侧并掉速。R044 装上按弯急缓分叉的逻辑后，一些弯确实不再切内，却换来了外偏，甚至蹭到外栏。R046 回到只看近处横偏的门控。这个结果反过来印证了上一段的判据选择。入弯瞬间唯一稳定可靠的信号就是近处横偏。

图 13 把三代门控画在同一张横偏到远处预瞄项放行比例的曲线上。横轴是近处横偏的绝对值，

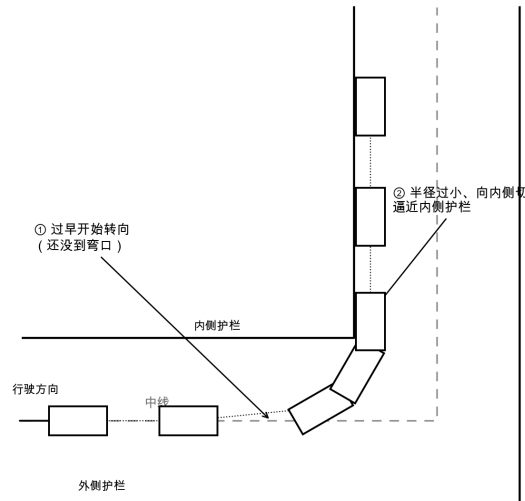
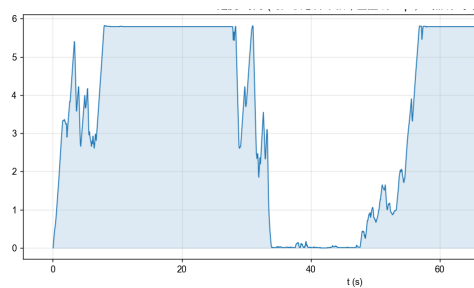
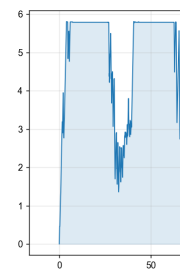


图 10: 转弯半径过小示意（顶视，90° 左转弯；矩形为车在各时刻的位置，仅为示意、非实测）。车本应沿中线（灰色虚线）行驶、到弯口附近再转向；实际却在还没到弯口时就开始打方向（1），以过小的半径切向弯内侧、逼近内侧护栏（2），而不是绕着中线走更大的半径。



(a) R026 在第一个左弯里速度掉到接近零、爬行十余秒（ $t \approx 34-47\text{s}$ ）；该轮在通过此弯后才于 $t \approx 93.6\text{s}$ 手动停止，末帧状态正常。



(b) R038 在同一个第一个左弯明显减速但不再掉到零（横轴是整圈 330s 中的前 65s）。

图 11: R026 与 R038 通过第一个左弯的速度-时间对比，两图裁到同一时间窗 $t \in [0, 65]\text{s}$ ，纵轴同为速度（0-6）。两轮只比较这第一个弯。R026 在弯中掉速到零，R038 明显减速但未再掉到零；R026 那一轮随后被手动停止，不与跑完整圈的 R038 比全程。

纵轴是远处预瞄项被放出的比例，曲线越早抬起，门就开得越早。旧门控（红线）在横偏为零时已开到约 0.94，几乎车一进入接近段就全部放行，这正是早切内的来源。纯横偏版（蓝线）改成一条线性曲线，要等横偏长到约 0.75 才完全放行，于是车会先沿直道开到弯口，横偏出现后才逐步加大转向。速度耦合版（紫线）则更陡，在高速下把完全放行对应的有效横偏基准压到约 0.43。高速时同样的时间内车已经多跑出一段，门必须相应提前开，这就引出下面的速度补强。

R047-R048 两轮补了两个剩余漏洞。其一是把车速引入门控基准（R047）。车速越高，横偏长起来之前车已经多滑出一段距离，若沿用低速时的门控，几何上同样晚、物理上却更晚，结果冲外、半径过大；所以高速时门要更早开。这里用车速而不是弯道缓急来调，因为车速在入弯瞬间就已稳定可读，而曲率风险此刻还不可靠（上一段的教训）。其二是给门加跨帧保持（R048）。横偏在弯中会短暂回落，若门跟着回落，远处预瞄项被收掉，车就会转到一半收轮；于是进弯后把放行比例锁到峰值，出弯再按衰减系数慢慢松开。

把同一个紧弯在四个重要版本上的数字摆在一起（图 14、表 10），能看到问题是一步步解决的，而不是一次调参就调好。R041 线偏移峰值 0.65，严重切内，十二点硬撞，均速仅 0.61；换成横偏判据的 R042 峰值降到 0.23，切内和硬撞一起消失；R044 进一步压到 0.02，横向几乎不动，但它对应

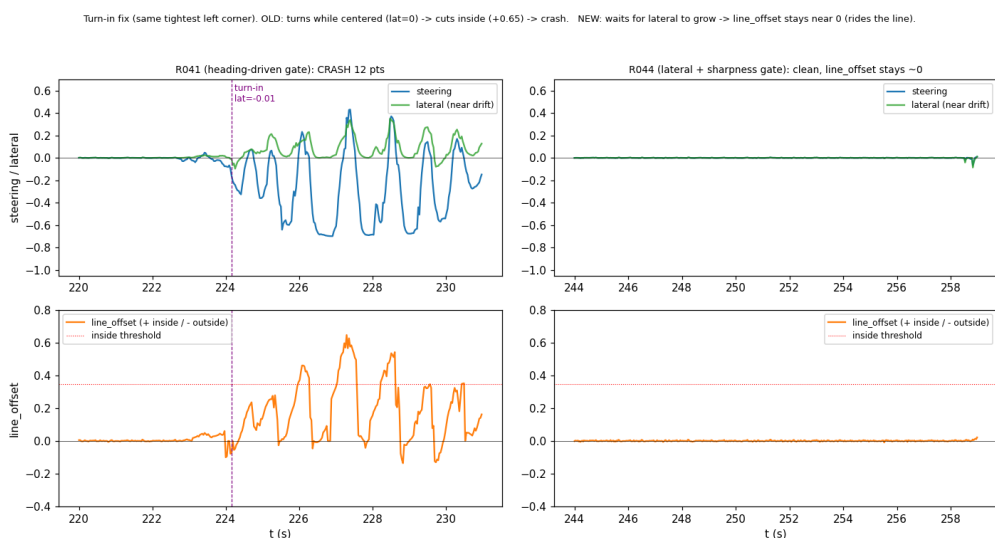


图 12: 同一最紧左弯的入弯对照。左列 (R041, 走向驱动门控) 在横偏约 -0.01 、车还居中时就开始入弯, 随后线偏移反复越过内侧阈值 0.35 、峰值 $+0.65$, 十二点硬撞; 右列为 R044 的横偏加弯急缓门控窗口, 线偏移基本贴近零。

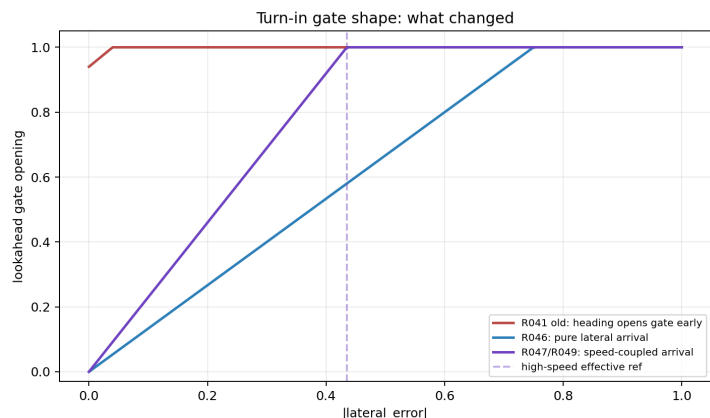


图 13: 三代入弯门控形状对比。红线 (旧) 横偏为零时已近全开; 蓝线 (纯横偏) 到 0.75 才全开; 紫线 (速度耦合) 在高速有效基准约 0.43 处全开。

按弯急缓分叉的版本, 后来因外偏被删; R049 的峰值看似又回到 0.79 , 但方向是向外的。车不再切入, 只剩轻微擦碰, 均速也提到 0.81 。峰值数字这番反复, 恰好对应判据更换、删掉错误分叉、速度耦合、门控保持这一连串机制改动, 而非反复拧同一个参数。

表 10: 紧弯控制迭代演进 (同一最紧弯窗口)

版本	线偏移峰值	均速	说明
R041	0.65	0.61	旧门控, 十二点硬撞
R042	0.23	0.75	改横偏判据, 硬撞消失
R044	0.02	0.95	按弯急缓分叉 (后因外偏删除)
R049	0.79	0.81	当前最佳, 峰值方向向外、仅轻擦

入弯修好后开始系统性提速。均速从 R044 的 0.62 提到 0.83 , 再到 R049 的 0.85 , 中位速度从约 0.66 升到 0.89 , 丢线率始终为零 (图 15 左)。提速靠几处改动叠加, 而不是简单调高限速。先放开早期过于保守的全局限速和直道速度; 再用前面那套速度耦合, 避免高速入弯因门开得太晚而冲外; 又靠门控保持消除弯后跑偏、再修正、再掉速这类隐性损失; 最后定向抬高中等弯的速度。最后一步靠 `curve_power` 这个旋钮 (图 15 右)。把它从默认值提到 1.5 后, 中等曲率弯的速度因子被

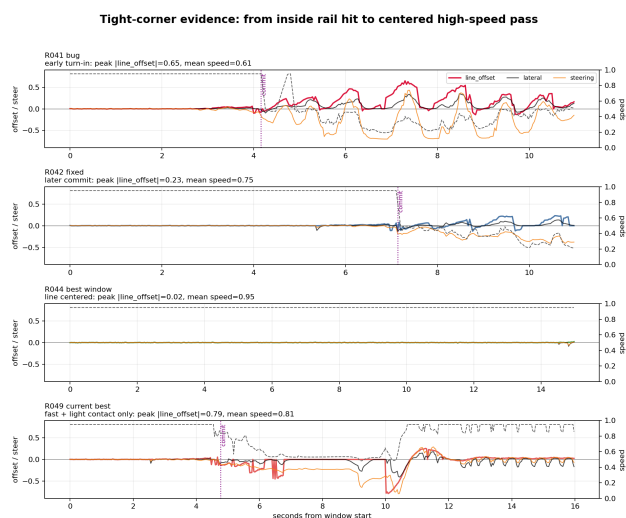
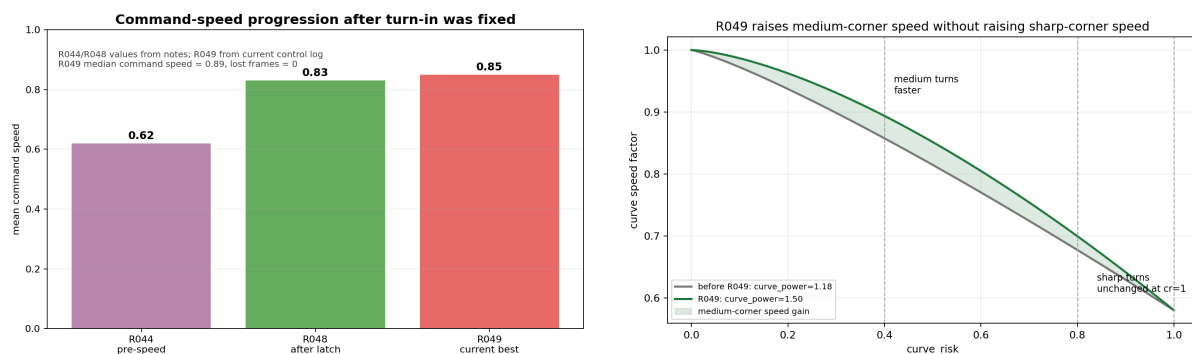


图 14: 同类紧弯控制量在四个版本上的演进（自上而下 R041 / R042 / R044 / R049）。

抬高，而曲率风险到顶的急弯端点保持不变，因为急弯已接近速度与半径的物理边界，再快只会走得更宽，甚至撞栏。



(a) R044 到 R049 均速进展 (0.62 → 0.83 → 0.85)。

(b) curve_power 提到 1.5 后只抬中等弯，急弯端点不变。

图 15: 提速的两条证据。整体均速抬升，且通过 curve_power 把提速集中在中等弯。

最后还剩一个没根除的问题，但根源不在门控，而在感知，即两个肉眼看着差不多的弯，有时会走出不一致的半径。原因是感知给出的曲率风险不够稳定，弯与弯之间忽高忽低，于是降速幅度不同，过弯的物理半径也就不同。图 16 是一个代表帧。这个右弯里左右相机各只检测到 3 个白线点，证据稀疏，纠正来得偏晚，车已落到白线内侧。第 6.1 节继续讨论这个感知一致性问题。

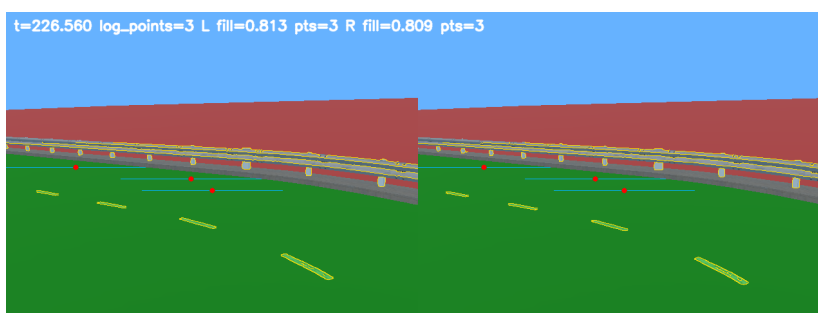


图 16: 残留窗口的双目叠加图。顶部日志显示左右相机各只有 3 个白线点；白线证据稀疏，纠正偏晚，车落到白线内侧。这是感知一致性问题，不是入弯门控问题。

6 不足与经验教训

6.1 不足与后续工作

本项目有一个来自传感器的硬限制。双目相机只朝前，控制器对车身两侧和正后方一无所知（`control(left_img, right_img, timestamp)` 每帧只有两幅前视图像，看不到与自己并排或在身后的对手，也无从估计其速度和意图）。这个盲区在多车场景反复制造麻烦。最直接的是发车时六辆同策略车挤在同一起步格里，但每辆车视野中只有前方空地、看不见身侧的邻车，于是谁也不会主动让位，发车一刻六车密集重叠、互有轻触（用第 3 节的顶视复盘回看开局即可看到车体交叠）。盲区也限制了行驶中的对抗。对手从侧后方贴上来并排时，本车要等它切到正前方才看见，此前无法预先让线或卡位，对身后的追赶者更无从防守。因此避让和走位再细，也只能作用于已出现在前方的对手；而多车竞速有大量博弈发生在侧后方，这基本是平台给定的硬上限。

即便对看得见的前方对手，当前策略也只是减速加朝开阔侧绕行，属于安全避让，谈不上对抗。避让链路（第 2.2.4 节）检测到近处对手后按其居中程度减速，并朝余量更大的一侧加一个有界转向偏置绕行。留有横向净空时，这套策略足够干净。摆位实验中（图 7）对手贴上侧停住、车道可通行，本车贴下侧绕过、不掉零完成超车。但它没有基于车位和相对速度的主动占位，例如卡内线逼对手走外线。净空一旦不够就暴露为失败。图 8 中对手斜停、几乎横跨整条 8m 车道，两车各约 2m 宽、净空不足，本车在距其约 3.8m 处被楔住，只能退化到 `force_escape` 倒车。更深的问题是决策太晚。控制器直到逼近、绕行失败后才被动脱困。把避让推进到主动占位、并把决策点前移是下一步，但它同样受制于前述感知盲区，因为占位和防守都需要侧后方信息。

经过转弯半径攻坚后，驾驶逻辑已较成熟，残余的走线误差几乎都能追溯到感知不稳。瓶颈已从控制转移到感知。门控、降速和分层脱困定型后，单车在 `complex` 已零丢线、零硬撞跑完整圈（R049 日志 8053 帧，满速帧占 47.5%、低速爬行仅 0.3%）。此时再出的问题大多是两个肉眼相近的弯走出不同半径。复盘这类窗口（图 16， $t \approx 226.56\text{s}$ ）会发现根因不在门控。该帧左右相机各只检测到 3 个白线点，曲率风险估计随之抖动，降速幅度和物理半径就跟着不一致。控制律对相同输入给出一致输出，但输入（感知量）在相近场景下不够稳定。强红光下掩膜饱和被误判成丢线、草地被并入路面（第 2.2.1 节）也是同类表现。因此下一阶段不应继续围绕控制参数微调，而应提高感知一致性。具体方向是提高不同光照路面下白线点的召回密度，并重新标定曲率信任度的动态范围。

6.2 经验教训

把系统拆成单向流水线、用少数固定的数据结构作为模块间唯一接口，是最划算的架构决定。它让每类异常都能定位到唯一模块。控制器按感知、估计、策略、限幅的顺序组织，模块只通过 `common.py` 的三个 `dataclass`（`PerceptionObs`、`TrackState`、`ControlCmd`）交换信息。`perception` 不算控制量，`policy` 不回读原图。走线异常时先看 `estimator` 的几何量是否合理。`TrackState` 里横偏、曲率本身就错，就查 `perception`；几何量正常而舵角不对，就查 `policy`。转弯半径那条主线正是这样定位的。对比同一帧的感知叠加图和日志里的 `target_steering`，确认看对了路、但门控过早放出远处预瞄项，矛头才指向 `policy`。把可调参数全部集中到 `params.py`、不在模块内硬编码，则让改参数与改逻辑分离。代价是契约很硬。改任一字段须同步所有调用方、构建脚本和测试，否则单文件在运行期就因字段不匹配报错；但这种改坏立刻报错远比静默出错安全。

要让两套策略互不污染，必须用代码路径开关做硬隔离，不能只靠参数设零所以不触发这种软约束。教训来自多车迭代早期（Stage 5，R053–R061）。为做避让和脱困，我们把对手减速、倒车、`force_escape` 和光流卡死检测直接加到当时唯一的一套参数上，等于把多车策略强加给了单车计时赛。后果是单车场景里这些行为仍会触发。光流卡死检测在某个急弯的减速窗口误判画面不动、触

发倒车摆轮；避让逻辑也会因画面干扰无谓减速转向，凭空拖慢成绩。根因是参数置零时分支代码仍在执行、跨帧状态（如卡死计数器）仍在累积，与其它逻辑交互后就可能越过阈值。修复方式是硬隔离。单车参数从多车字典派生、覆盖掉所有多车增量（第 2.2.5 节），并在管线层用 `profile` 开关决定是否运行对手检测和 `force_escape`，使单车彻底不走多车代码路径（每帧顺带省约 5 ms）；再用 `test_profile_isolation.py` 锁定这条约束。安全边界必须由代码结构和测试保证，不能交给软约束。

在拿不到真值、平台反馈又少的任务里，先解决怎么判断好不好，比急着调策略更重要；这次的弯路集中在度量上。其一，离线统计量会骗人，丢线率尤甚。掩膜饱和会被记成丢线，草地并入路面也会扭曲它，单看会得出与实跑相反的结论，所以驾驶质量最终要靠接触日志和逐帧叠加图判断。其二，平台默认看不见撞栏。`telemetry` 只记车车碰撞、撞护栏一概不报，我们因此在 SDK 的 `supervisor` 层注入接触检测（第 3 节），撞没撞、撞在哪、多重才有了离线证据。其三，多车主要目标长期无法度量。检查点坐标是占位值、触发门仅 0.1 m 厚（车以约 6 m/s 通过时一帧就跨过整个门）、初始序号还错位，完赛和名次根本统计不出来；按真实坐标重写、把门加厚到约 2.5 m、修正序号后才有可信基准。共同教训是，度量不可信时，基于它的优化都站不住，应先把取证和度量流程搭起来。

在耦合很强的控制系统里，见效的改进往往是结构性的，而不是反复拧参数。转弯半径那条主线（R026-R049）是最清楚的例证。早期也在走向驱动门控上反复调阈值，但只要判据仍是远处走向与曲率，车就一定在直道接近段被误判成已到弯口、提前切内；参数怎么调都只是换种方式暴露同一个原理缺陷。解决问题靠三处结构改动。把判据从走向换成近处横偏，删掉按弯急缓分叉那条反而引入外偏的逻辑，再给门控补上速度耦合与跨帧保持。脱困同理。把单一的低速检测换成按优先级排列的四层自救链，才把最长卡死从十余秒压到约两秒，而不是靠调某个阈值。这条经验改变了工作方式。遇到反复调参却收效甚微的瓶颈，应先怀疑判据来源或结构本身选错，转而重构，而不是在错误结构里继续搜参数。